FastAPI Backend Implementation Plan for GTD Search App

Project Overview

- **Data Source**: dbt project with TickTick todo data in Postgres
- **Dataset Size**: ~8,000 records, single user
- **Time Constraint**: 4-8 hours per week development time
- **Key Feature**: Text search using PostgreSQL's full-text search capabilities
- **Future Enhancement**: Vector search for AI agent integration

Database Schema

The main table we're exposing is `fact_todos` which contains:

- Todo metadata (title, content, status)
- Relationship data (list name, folder name)
- Date information (due date, created date, etc.)
- Text search vector column created by the `setup_textsearch` macro

Implementation Phases

Phase 1: Core Search API (4-6 weeks)

1. Database Connection Setup

```
from sqlalchemy import create_engine, MetaData
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from sqlmodel import SQLModel, Field, Session
from typing import Generator
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()
```

```
DATABASE_URL = os.getenv("DATABASE_URL",
"postgresql://username:password@localhost:5432/ticktick_db")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
metadata = MetaData()

# Function to get DB session
def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

2. Basic Models

```
from sqlalchemy import Column, String, DateTime, Boolean, Integer, Float
from sqlmodel import SQLModel, Field
from typing import Optional, List
from datetime import datetime
from pydantic import BaseModel

# Basic Todo model for responses
class Todo(BaseModel):
    todo_id: str
    todo_title: str
    todo_content: Optional[str] = None
    todo_list_name: Optional[str] = None
    todo_folder_name: Optional[str] = None
    todo_status: Optional[str] = None
    todo_duedate: Optional[datetime] = None
    todo_tags: Optional[str] = None
    created_at: Optional[datetime] = None
    updated_at: Optional[datetime] = None

# Search parameters model
class TodoSearch(BaseModel):
    query: Optional[str] = None
    list_name: Optional[str] = None
    folder_name: Optional[str] = None
    status: Optional[str] = None
    due_date_start: Optional[datetime] = None
    due_date_end: Optional[datetime] = None
    completed: Optional[bool] = None
    page: int = 1
    page_size: int = 20
```

3. Core Search Endpoint

```python
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy import text
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import datetime

from app.database import get_db
from app.models import Todo, TodoSearch

router = APIRouter()

@router.get("/todos/search", response_model=List[Todo])
def search_todos(
    search_params: TodoSearch = Depends(),
    db: Session = Depends(get_db)
):
    # Start with a base query
    query = """
    SELECT * FROM marts_core.fact_todos
    WHERE 1=1
    """
    params = {}

    # Add text search if query is provided
    if search_params.query:
        query += """
        AND search @@ websearch_to_tsquery('english', :query)
        ORDER BY ts_rank(search, websearch_to_tsquery('english', :query))
DESC
        """
        params["query"] = search_params.query

    # Add other filters
    if search_params.list_name:
        query += " AND todo_list_name = :list_name"
        params["list_name"] = search_params.list_name

    if search_params.folder_name:
        query += " AND todo_folder_name = :folder_name"
        params["folder_name"] = search_params.folder_name

    if search_params.status:
        query += " AND status_id = :status"
        params["status"] = search_params.status

    if search_params.due_date_start:
        query += " AND todo_duedate >= :due_date_start"
        params["due_date_start"] = search_params.due_date_start

    if search_params.due_date_end:
        query += " AND todo_duedate <= :due_date_end"
        params["due_date_end"] = search_params.due_date_end

    if search_params.completed is not None:
        query += " AND todo_status = :completed"
        params["completed"] = "0" if not search_params.completed else "1"
```

```
    # Add pagination
    query += " LIMIT :limit OFFSET :offset"
    params["limit"] = search_params.page_size
    params["offset"] = (search_params.page - 1) * search_params.page_size

    result = db.execute(text(query), params).fetchall()

    # Convert to dictionaries
    return [dict(row._mapping) for row in result]

@router.get("/todos/{todo_id}", response_model=Todo)
def get_todo(todo_id: str, db: Session = Depends(get_db)):
    result = db.execute(
        text("SELECT * FROM marts_core.fact_todos WHERE todo_id =
:todo_id"),
        {"todo_id": todo_id}
    ).first()

    if not result:
        raise HTTPException(status_code=404, detail="Todo not found")

    return dict(result._mapping)
```

4. Main FastAPI App

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.api import todos

app = FastAPI(title="GTD Search API")

# CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Adjust for production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Include routers
app.include_router(todos.router, prefix="/api", tags=["todos"])

@app.get("/")
def read_root():
    return {"message": "Welcome to GTD Search API"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("app.main:app", host="0.0.0.0", port=8000, reload=True)
```

Phase 2: Enhanced Search & Metadata (4-6 weeks)

1. Metadata Endpoints

```
@router.get("/folders", response_model=List[str])
def get_folders(db: Session = Depends(get_db)):
    result = db.execute(
        text("SELECT DISTINCT todo_folder_name FROM marts_core.fact_todos
ORDER BY todo_folder_name")
    ).fetchall()

    return [row[0] for row in result if row[0]]

@router.get("/lists", response_model=List[str])
def get_lists(db: Session = Depends(get_db)):
    result = db.execute(
        text("SELECT DISTINCT todo_list_name FROM marts_core.fact_todos
ORDER BY todo_list_name")
    ).fetchall()

    return [row[0] for row in result if row[0]]

@router.get("/tags", response_model=List[str])
def get_tags(db: Session = Depends(get_db)):
    # This assumes tags are stored as a comma-separated string
    result = db.execute(
        text("SELECT DISTINCT unnest(string_to_array(todo_tags, ',')) as tag
FROM marts_core.fact_todos")
    ).fetchall()

    return [row[0] for row in result if row[0]]

@router.get("/statuses", response_model=List[str])
def get_statuses(db: Session = Depends(get_db)):
    result = db.execute(
        text("SELECT DISTINCT todo_status FROM marts_core.fact_todos")
    ).fetchall()

    return [row[0] for row in result if row[0]]
```

2. Enhanced Search Model

```
class EnhancedTodoSearch(TodoSearch):
    tags: Optional[List[str]] = None
    priority: Optional[List[str]] = None
    sort_by: Optional[str] = None
    sort_direction: Optional[str] = "desc"
    search_title: Optional[bool] = True
```

```
    search_content: Optional[bool] = True
    search_metadata: Optional[bool] = True
    search_tags: Optional[bool] = True
```

3. Enhanced Search Endpoint

```
@router.get("/todos/search/enhanced", response_model=List[Todo])
def enhanced_search_todos(
    search_params: EnhancedTodoSearch = Depends(),
    db: Session = Depends(get_db)
):
    # Start with a base query
    query = """
    SELECT * FROM marts_core.fact_todos
    WHERE 1=1
    """
    params = {}

    # Add text search if query is provided
    if search_params.query:
        search_conditions = []

        if search_params.search_title:
            search_conditions.append("todo_title ILIKE :title_query")
            params["title_query"] = f"%{search_params.query}%"

        if search_params.search_content:
            search_conditions.append("todo_content ILIKE :content_query")
            params["content_query"] = f"%{search_params.query}%"

        if search_params.search_metadata:
            search_conditions.append("todo_list_name ILIKE :list_query")
            search_conditions.append("todo_folder_name ILIKE :folder_query")
            params["list_query"] = f"%{search_params.query}%"
            params["folder_query"] = f"%{search_params.query}%"

        if search_params.search_tags:
            search_conditions.append("todo_tags ILIKE :tags_query")
            params["tags_query"] = f"%{search_params.query}%"

        if search_conditions:
            query += " AND (" + " OR ".join(search_conditions) + ")"

        # Also use the full-text search for better ranking
        query += """
        ORDER BY ts_rank(search, websearch_to_tsquery('english',
:full_query)) DESC
        """
        params["full_query"] = search_params.query

    # Add other filters (same as basic search)
    # ...

    # Add tag filtering
```

```
    if search_params.tags and len(search_params.tags) > 0:
        tag_conditions = []
        for i, tag in enumerate(search_params.tags):
            tag_param = f"tag_{i}"
            tag_conditions.append(f"todo_tags ILIKE :{tag_param}")
            params[tag_param] = f"%{tag}%"

        query += " AND (" + " OR ".join(tag_conditions) + ")"

    # Add priority filtering
    if search_params.priority and len(search_params.priority) > 0:
        priority_conditions = []
        for i, priority in enumerate(search_params.priority):
            priority_param = f"priority_{i}"
            priority_conditions.append(f"todo_priority = :{priority_param}")
            params[priority_param] = priority

        query += " AND (" + " OR ".join(priority_conditions) + ")"

    # Add sorting
    if search_params.sort_by:
        query += f" ORDER BY {search_params.sort_by}
{search_params.sort_direction}"
    elif not search_params.query:  # If no text search, default sort by due
date
        query += " ORDER BY todo_duedate DESC"

    # Add pagination
    query += " LIMIT :limit OFFSET :offset"
    params["limit"] = search_params.page_size
    params["offset"] = (search_params.page - 1) * search_params.page_size

    result = db.execute(text(query), params).fetchall()

    return [dict(row._mapping) for row in result]
```

Phase 3: Workspace Management (4-6 weeks)

1. Workspace Models

```
class PaneGroup(BaseModel):
    id: str
    task_ids: List[str]
    active_task_id: Optional[str] = None
    size: Optional[int] = None

class Workspace(BaseModel):
    id: str
    user_id: str
    name: str
    open_tasks: List[str]
    pane_groups: List[PaneGroup]
```

```
    active_task_id: Optional[str] = None
    created_at: datetime = Field(default_factory=datetime.now)
    updated_at: datetime = Field(default_factory=datetime.now)
```

2. Workspace Endpoints

```
@router.post("/workspaces", response_model=Workspace)
def create_workspace(
    workspace: Workspace,
    db: Session = Depends(get_db)
):
    # Convert workspace to JSON
    workspace_json = workspace.json()

    # Insert into database
    result = db.execute(
        text("""
        INSERT INTO user_workspaces (id, user_id, name, workspace_data,
created_at, updated_at)
        VALUES (:id, :user_id, :name, :workspace_data, :created_at,
:updated_at)
        RETURNING id
        """),
        {
            "id": workspace.id,
            "user_id": workspace.user_id,
            "name": workspace.name,
            "workspace_data": workspace_json,
            "created_at": workspace.created_at,
            "updated_at": workspace.updated_at
        }
    )

    db.commit()

    return workspace

@router.get("/workspaces/{workspace_id}", response_model=Workspace)
def get_workspace(
    workspace_id: str,
    db: Session = Depends(get_db)
):
    result = db.execute(
        text("SELECT workspace_data FROM user_workspaces WHERE id = :id"),
        {"id": workspace_id}
    ).first()

    if not result:
        raise HTTPException(status_code=404, detail="Workspace not found")

    return Workspace.parse_raw(result[0])

@router.put("/workspaces/{workspace_id}", response_model=Workspace)
def update_workspace(
```

```
    workspace_id: str,
    workspace: Workspace,
    db: Session = Depends(get_db)
):
    # Update timestamp
    workspace.updated_at = datetime.now()

    # Convert workspace to JSON
    workspace_json = workspace.json()

    # Update in database
    result = db.execute(
        text("""
        UPDATE user_workspaces
        SET workspace_data = :workspace_data, updated_at = :updated_at
        WHERE id = :id
        RETURNING id
        """),
        {
            "id": workspace_id,
            "workspace_data": workspace_json,
            "updated_at": workspace.updated_at
        }
    )

    if not result.first():
        raise HTTPException(status_code=404, detail="Workspace not found")

    db.commit()

    return workspace

@router.delete("/workspaces/{workspace_id}", status_code=204)
def delete_workspace(
    workspace_id: str,
    db: Session = Depends(get_db)
):
    result = db.execute(
        text("DELETE FROM user_workspaces WHERE id = :id"),
        {"id": workspace_id}
    )

    if result.rowcount == 0:
        raise HTTPException(status_code=404, detail="Workspace not found")

    db.commit()

    return None
```

Phase 4: Vector Search Integration (4-6 weeks)

1. Setup pgvector Extension

```
-- Run this in PostgreSQL to enable vector search
CREATE EXTENSION IF NOT EXISTS vector;

-- Add vector column to the todos table
ALTER TABLE marts_core.fact_todos
ADD COLUMN IF NOT EXISTS embedding vector(1536);

-- Create an index for the embedding column
CREATE INDEX IF NOT EXISTS idx_todos_embedding ON marts_core.fact_todos
USING ivfflat (embedding vector_l2_ops) WITH (lists = 100);
```

2. Embedding Generation Service

```
import openai
import os
import numpy as np
from typing import List, Dict, Any
from sqlalchemy import text
from sqlalchemy.orm import Session

class EmbeddingService:
    def __init__(self):
        openai.api_key = os.getenv("OPENAI_API_KEY")
        self.model = "text-embedding-ada-002"
        self.embedding_dims = 1536

    def generate_embedding(self, text: str) -> List[float]:
        """Generate embedding for a single text string"""
        if not text or text.strip() == "":
            # Return zero vector for empty text
            return [0.0] * self.embedding_dims

        response = openai.Embedding.create(
            input=text,
            model=self.model
        )

        return response["data"][0]["embedding"]

    def generate_task_embedding(self, task: Dict[str, Any]) -> List[float]:
        """Generate embedding for a task by combining title, content, and
metadata"""
        # Combine relevant fields with appropriate weighting
        combined_text = f"Title: {task.get('todo_title', '')} "
        combined_text += f"Content: {task.get('todo_content', '')} "
        combined_text += f"List: {task.get('todo_list_name', '')} "
        combined_text += f"Folder: {task.get('todo_folder_name', '')} "
        combined_text += f"Tags: {task.get('todo_tags', '')}"
```

```
        return self.generate_embedding(combined_text)

    def update_task_embedding(self, task_id: str, db: Session) -> bool:
        """Update the embedding for a specific task"""
        # Get the task data
        result = db.execute(
            text("SELECT * FROM marts_core.fact_todos WHERE todo_id =
:todo_id"),
            {"todo_id": task_id}
        ).first()

        if not result:
            return False

        task = dict(result._mapping)

        # Generate the embedding
        embedding = self.generate_task_embedding(task)

        # Update the database
        db.execute(
            text("UPDATE marts_core.fact_todos SET embedding = :embedding
WHERE todo_id = :todo_id"),
            {"todo_id": task_id, "embedding": embedding}
        )

        db.commit()
        return True

    def update_all_embeddings(self, db: Session, batch_size: int = 100) ->
int:
        """Update embeddings for all tasks in batches"""
        # Get all task IDs
        result = db.execute(
            text("SELECT todo_id FROM marts_core.fact_todos WHERE embedding
IS NULL")
        ).fetchall()

        task_ids = [row[0] for row in result]
        updated_count = 0

        # Process in batches
        for i in range(0, len(task_ids), batch_size):
            batch = task_ids[i:i+batch_size]
            for task_id in batch:
                if self.update_task_embedding(task_id, db):
                    updated_count += 1

        return updated_count
```

3. Vector Search Endpoint

```
class VectorSearchParams(BaseModel):
    query: str
```

```
    filter_list_name: Optional[str] = None
    filter_folder_name: Optional[str] = None
    filter_status: Optional[str] = None
    filter_completed: Optional[bool] = None
    hybrid_search: bool = True
    similarity_threshold: float = 0.7
    page: int = 1
    page_size: int = 20

@router.post("/todos/vector-search", response_model=List[Todo])
def vector_search_todos(
    search_params: VectorSearchParams,
    db: Session = Depends(get_db),
    embedding_service: EmbeddingService = Depends(lambda:
EmbeddingService())
):
    # Generate embedding for the query
    query_embedding =
embedding_service.generate_embedding(search_params.query)

    # Build the query
    query = """
    SELECT *,
           1 - (embedding <-> :query_embedding) AS similarity
    FROM marts_core.fact_todos
    WHERE 1=1
    """

    params = {"query_embedding": query_embedding}

    # Add filters
    if search_params.filter_list_name:
        query += " AND todo_list_name = :list_name"
        params["list_name"] = search_params.filter_list_name

    if search_params.filter_folder_name:
        query += " AND todo_folder_name = :folder_name"
        params["folder_name"] = search_params.filter_folder_name

    if search_params.filter_status:
        query += " AND todo_status = :status"
        params["status"] = search_params.filter_status

    if search_params.filter_completed is not None:
        query += " AND todo_status = :completed"
        params["completed"] = "0" if not search_params.filter_completed else
"1"

    # Add similarity threshold
    query += " AND 1 - (embedding <-> :query_embedding) > :threshold"
    params["threshold"] = search_params.similarity_threshold

    # Add hybrid search if requested
    if search_params.hybrid_search and search_params.query:
        query += """
        ORDER BY
            (1 - (embedding <-> :query_embedding)) * 0.7 +
```

```
            ts_rank(search, websearch_to_tsquery('english', :text_query)) *
0.3
            DESC
        """
        params["text_query"] = search_params.query
    else:
        query += " ORDER BY similarity DESC"

    # Add pagination
    query += " LIMIT :limit OFFSET :offset"
    params["limit"] = search_params.page_size
    params["offset"] = (search_params.page - 1) * search_params.page_size

    result = db.execute(text(query), params).fetchall()

    # Convert to list of dictionaries with similarity score
    todos = []
    for row in result:
        todo_dict = dict(row._mapping)
        # Remove the embedding vector from the response
        if "embedding" in todo_dict:
            del todo_dict["embedding"]
        todos.append(todo_dict)

    return todos

# Endpoint to trigger embedding generation
@router.post("/admin/update-embeddings", response_model=Dict[str, int])
def update_embeddings(
    batch_size: int = Query(100, description="Number of tasks to process in
each batch"),
    db: Session = Depends(get_db),
    embedding_service: EmbeddingService = Depends(lambda:
EmbeddingService())
):
    count = embedding_service.update_all_embeddings(db, batch_size)
    return {"updated_count": count}
```

4. Hybrid Search Implementation

```
class HybridSearchParams(BaseModel):
    query: str
    vector_weight: float = 0.7
    text_weight: float = 0.3
    filters: Optional[Dict[str, Any]] = None
    page: int = 1
    page_size: int = 20

@router.post("/todos/hybrid-search", response_model=List[Todo])
def hybrid_search_todos(
    search_params: HybridSearchParams,
    db: Session = Depends(get_db),
    embedding_service: EmbeddingService = Depends(lambda:
EmbeddingService())
```

```
):
    # Generate embedding for the query
    query_embedding =
embedding_service.generate_embedding(search_params.query)

    # Build the query with both vector similarity and text search
    query = """
    SELECT *,
           (:vector_weight * (1 - (embedding <-> :query_embedding))) +
           (:text_weight * ts_rank(search, websearch_to_tsquery('english',
:text_query)))
           AS combined_score
    FROM marts_core.fact_todos
    WHERE 1=1
    """

    params = {
        "query_embedding": query_embedding,
        "text_query": search_params.query,
        "vector_weight": search_params.vector_weight,
        "text_weight": search_params.text_weight
    }

    # Add filters
    if search_params.filters:
        for key, value in search_params.filters.items():
            if value is not None:
                if key == "list_name":
                    query += " AND todo_list_name = :list_name"
                    params["list_name"] = value
                elif key == "folder_name":
                    query += " AND todo_folder_name = :folder_name"
                    params["folder_name"] = value
                elif key == "status":
                    query += " AND todo_status = :status"
                    params["status"] = value
                elif key == "completed":
                    query += " AND todo_status = :completed"
                    params["completed"] = "0" if not value else "1"
                elif key == "tags":
                    if isinstance(value, list) and len(value) > 0:
                        tag_conditions = []
                        for i, tag in enumerate(value):
                            tag_param = f"tag_{i}"
                            tag_conditions.append(f"todo_tags ILIKE :
{tag_param}")
                            params[tag_param] = f"%{tag}%"
                        query += " AND (" + " OR ".join(tag_conditions) +
")"

    # Order by combined score
    query += " ORDER BY combined_score DESC"

    # Add pagination
    query += " LIMIT :limit OFFSET :offset"
    params["limit"] = search_params.page_size
    params["offset"] = (search_params.page - 1) * search_params.page_size
```

```
    result = db.execute(text(query), params).fetchall()

    # Convert to list of dictionaries
    todos = []
    for row in result:
        todo_dict = dict(row._mapping)
        # Remove the embedding vector from the response
        if "embedding" in todo_dict:
            del todo_dict["embedding"]
        todos.append(todo_dict)

    return todos
```

Project Structure

```
project/
├── app/
│   ├── __init__.py
│   ├── main.py                 # FastAPI app
│   ├── database.py             # DB connection
│   ├── models/
│   │   ├── __init__.py
│   │   ├── todo.py             # Todo models
│   │   └── workspace.py        # Workspace models
│   ├── services/
│   │   ├── __init__.py
│   │   └── embedding.py        # Embedding generation service
│   ├── api/
│   │   ├── __init__.py
│   │   ├── todos.py            # Todo endpoints
│   │   ├── metadata.py         # Metadata endpoints
│   │   ├── workspaces.py       # Workspace endpoints
│   │   └── vector_search.py    # Vector search endpoints
│   └── utils/
│       ├── __init__.py
│       └── search.py           # Search utilities
├── alembic/                    # Database migrations
│   ├── versions/
│   ├── env.py
│   └── alembic.ini
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_todos.py
│   └── test_vector_search.py
├── .env                        # Environment variables
├── .gitignore
├── requirements.txt
├── Dockerfile
└── docker-compose.yml
```

Requirements

```
fastapi>=0.68.0
uvicorn>=0.15.0
sqlmodel>=0.0.4
sqlalchemy>=1.4.0
psycopg2-binary>=2.9.1
python-dotenv>=0.19.0
openai>=0.27.0
numpy>=1.20.0
pytest>=6.0.0
httpx>=0.18.0
python-jose>=3.3.0
passlib>=1.7.4
alembic>=1.7.0
```

Implementation Timeline

Phase 1: Core Search API (Weeks 1-6)

- Week 1-2: Set up project structure, database connection
- Week 3-4: Implement basic search endpoint
- Week 5-6: Add basic filtering and pagination

Phase 2: Enhanced Search & Metadata (Weeks 7-12)

- Week 7-8: Implement metadata endpoints
- Week 9-10: Enhance search with additional filters
- Week 11-12: Add sorting and grouping capabilities

Phase 3: Workspace Management (Weeks 13-18)

- Week 13-14: Create workspace models and database schema

- Week 15-16: Implement workspace CRUD operations
- Week 17-18: Add workspace state persistence

Phase 4: Vector Search Integration (Weeks 19-24)

- Week 19-20: Set up pgvector extension and embedding generation
- Week 21-22: Implement vector search endpoint
- Week 23-24: Create hybrid search capabilities

Future Enhancements

1. **AI Integration**
    - Expose the API through MCP for AI agent access
    - Implement natural language query understanding
    - Add task summarization and categorization
2. **Performance Optimizations**
    - Implement caching for frequent searches
    - Add query result pagination with cursors
    - Optimize vector search with approximate nearest neighbors
3. **Advanced Features**
    - Implement task relationships (parent/child, dependencies)
    - Add support for attachments and rich content
    - Create analytics endpoints for task patterns and productivity insights

Context for Future Reference

Text Search Implementation

The current implementation uses PostgreSQL's full-text search capabilities through a custom macro that:

1. Creates a custom text search configuration without stopwords
2. Adds a `search` tsvector column to the table
3. Creates a GIN index on this column
4. Sets up weights for different fields (title, content, list name, etc.)

This approach provides good performance for the current dataset size but has limitations for semantic understanding.

Vector Search Enhancement

The vector search implementation will:

1. Use the pgvector extension to store embeddings
2. Generate embeddings using OpenAI's text-embedding-ada-002 model
3. Combine vector similarity with traditional text search for hybrid results
4. Allow tuning of the weights between vector and text search

This approach will enable:

- Semantic understanding of tasks
- Finding similar tasks even with different terminology
- Better integration with AI agents
- More natural language query capabilities

API Design Philosophy

The API is designed with these principles:

1. **Progressive enhancement**: Basic functionality works without vector search
2. **Separation of concerns**: Search logic is separated from data access
3. **Flexibility**: Multiple search endpoints for different use cases
4. **Performance**: Efficient queries with proper indexing
5. **Extensibility**: Easy to add new search features or filters

Frontend Integration

The API is designed to support the frontend requirements outlined in FE_plan.md:

1. Advanced search interface with filtering
2. Workspace management with panels and tabs
3. Task detail views with rich content
4. Context preservation between sessions

The phased implementation allows the frontend to be developed in parallel, with each phase adding support for more advanced frontend features.